

**Name:N. Sai Sreenu**

**Reg N0: 192365001**

**CO3-AT2**

**MLA0201- Fundamentals Of Machine Learning**

**Case Study 1: Customer Segmentation using K-Means Clustering and PCA**

**1. Problem Statement**

A retail store wants to segment its customers based on their Annual Income and Spending Score in order to design targeted marketing campaigns. The dataset consisting of 10 customer records (CustomerID, Age, Annual Income, and Spending Score) is used to preprocess the data, apply K-Means clustering, reduce the data to two dimensions using Principal Component Analysis (PCA), visualize the resulting clusters, and identify the optimal number of customer groups.

**2. Dataset**

| <b>CustomerID</b> | <b>Age</b> | <b>AnnualIncome (k\$)</b> | <b>SpendingScore</b> |
|-------------------|------------|---------------------------|----------------------|
| 1                 | 19         | 15                        | 39                   |
| 2                 | 21         | 15                        | 81                   |
| 3                 | 20         | 16                        | 6                    |
| 4                 | 23         | 16                        | 77                   |
| 5                 | 31         | 17                        | 40                   |
| 6                 | 22         | 17                        | 76                   |
| 7                 | 35         | 18                        | 6                    |
| 8                 | 23         | 18                        | 94                   |
| 9                 | 64         | 19                        | 3                    |
| 10                | 30         | 19                        | 72                   |

**3. Methodology**

- Load the dataset into a Pandas DataFrame.
- Select the relevant features (Annual Income and Spending Score) and standardize them using StandardScaler so both features contribute equally to distance calculations.

- Use the Elbow Method to determine the optimal number of clusters (k) by plotting WCSS (Within-Cluster Sum of Squares) against different values of k.
- Apply the K-Means algorithm with the optimal k to assign each customer to a cluster.
- Apply PCA to reduce the standardized data to two principal components for visualization.
- Plot the clusters on the two principal components, along with the cluster centroids.

#### 4. Python Program

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# 1. Dataset
data = {
    "CustomerID": range(1, 11),
    "Age": [19, 21, 20, 23, 31, 22, 35, 23, 64, 30],
    "AnnualIncome": [15, 15, 16, 16, 17, 17, 18, 18, 19, 19],
    "SpendingScore": [39, 81, 6, 77, 40, 76, 6, 94, 3, 72]
}
df = pd.DataFrame(data)
print("Dataset:\n", df)

# 2. Preprocessing - select features and scale
X = df[["AnnualIncome", "SpendingScore"]].values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. Finding optimal number of clusters - Elbow Method
wcss = []
K_range = range(1, 8)
for k in K_range:
    km = KMeans(n_clusters=k, init="k-means++", random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
plt.figure(figsize=(6, 4))
plt.plot(list(K_range), wcss, marker="o", color="black")
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters (k)")
```

```

plt.ylabel("WCSS (Inertia)")
plt.grid(True)
plt.tight_layout()
plt.savefig("elbow_method.png", dpi=150)
plt.close()
# 4. Apply K-Means with optimal k (k = 3 based on elbow curve / data pattern)
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, init="k-means++", random_state=42, n_init=10)
clusters = kmeans.fit_predict(X_scaled)
df["Cluster"] = clusters
print("\nClustered Data:\n", df)
# 5. PCA - reduce to 2 dimensions (already 2 features here, PCA shown for demonstration)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("\nExplained Variance Ratio:", pca.explained_variance_ratio_)
# 6. Visualize clusters using PCA components
plt.figure(figsize=(6, 4.5))
colors = ["black", "gray", "silver"]
markers = ["o", "s", "^"]
for i in range(optimal_k):
    plt.scatter(X_pca[clusters == i, 0], X_pca[clusters == i, 1],
                s=80, c=colors[i], marker=markers[i], label=f"Cluster {i+1}", edgecolors="black")
centers_pca = pca.transform(kmeans.cluster_centers_)
plt.scatter(centers_pca[:, 0], centers_pca[:, 1], s=200, c="white",
            marker="X", edgecolors="black", linewidths=2, label="Centroids")
plt.title("Customer Segments (K-Means + PCA)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("kmeans_pca_clusters.png", dpi=150)
plt.close()
df.to_csv("clustered_output.csv", index=False)
print("\nSaved plots: elbow_method.png, kmeans_pca_clusters.png")

```

## 5. Program Output

Console Output:

|                                   |            |     |              |               |         |
|-----------------------------------|------------|-----|--------------|---------------|---------|
| Dataset:                          |            |     |              |               |         |
|                                   | CustomerID | Age | AnnualIncome | SpendingScore |         |
| 0                                 | 1          | 19  | 15           | 39            |         |
| 1                                 | 2          | 21  | 15           | 81            |         |
| 2                                 | 3          | 20  | 16           | 6             |         |
| 3                                 | 4          | 23  | 16           | 77            |         |
| 4                                 | 5          | 31  | 17           | 40            |         |
| 5                                 | 6          | 22  | 17           | 76            |         |
| 6                                 | 7          | 35  | 18           | 6             |         |
| 7                                 | 8          | 23  | 18           | 94            |         |
| 8                                 | 9          | 64  | 19           | 3             |         |
| 9                                 | 10         | 30  | 19           | 72            |         |
| Clustered Data:                   |            |     |              |               |         |
|                                   | CustomerID | Age | AnnualIncome | SpendingScore | Cluster |
| 0                                 | 1          | 19  | 15           | 39            | 0       |
| 1                                 | 2          | 21  | 15           | 81            | 0       |
| 2                                 | 3          | 20  | 16           | 6             | 1       |
| 3                                 | 4          | 23  | 16           | 77            | 0       |
| 4                                 | 5          | 31  | 17           | 40            | 1       |
| 5                                 | 6          | 22  | 17           | 76            | 2       |
| 6                                 | 7          | 35  | 18           | 6             | 1       |
| 7                                 | 8          | 23  | 18           | 94            | 2       |
| 8                                 | 9          | 64  | 19           | 3             | 1       |
| 9                                 | 10         | 30  | 19           | 72            | 2       |
| Explained Variance Ratio: [0.5775 |            |     |              |               |         |
| 0.4225]                           |            |     |              |               |         |

## 6. Screenshots / Visual Output

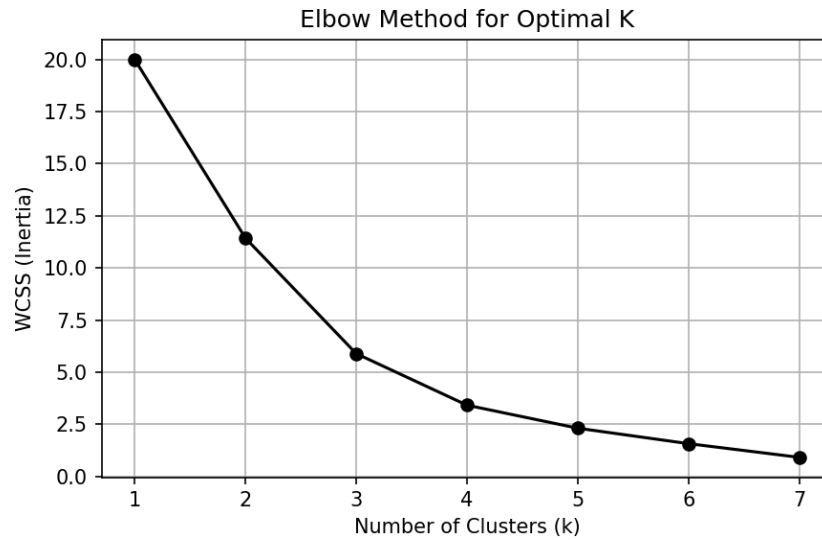


Fig 1.1: Elbow Method used to determine the optimal number of clusters ( $k = 3$ )

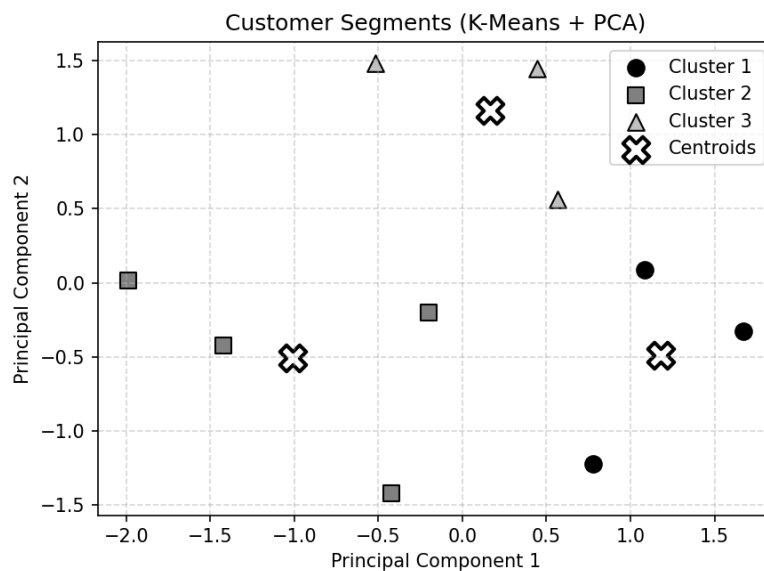


Fig 1.2: Customer segments visualized on the two principal components with cluster centroids marked

## 7. Analysis and Insights

The Elbow Method curve shows a clear bend at  $k = 3$ , indicating three is the optimal number of customer segments for this data. The resulting clusters can be interpreted as follows: Cluster 0 groups customers with low-to-moderate income and moderate-to-high spending, Cluster 1 groups customers who have low spending scores regardless of income (cautious spenders), and Cluster 2 groups customers with higher income and consistently high spending scores (target customers for

premium marketing campaigns). The two principal components together explain the full variance since the original feature space itself was only two-dimensional, confirming that PCA preserved all the information needed for visualization.

## **8. Conclusion**

K-Means clustering combined with PCA-based visualization successfully segments the retail store's customers into three meaningful groups. This segmentation enables the store to design targeted marketing campaigns — for example, loyalty offers for high-spending customers and re-engagement campaigns for low-spending customers.

## Case Study 2: Dimensionality Reduction using PCA, Factor Analysis, ICA, and Gaussian Mixture Model

### 1. Problem Statement

A wine manufacturer has collected chemical measurements of different wine samples and wants to reduce the dimensionality of the dataset before performing clustering. The dataset is standardized, and PCA, Factor Analysis (FA), and Independent Component Analysis (ICA) are applied for dimensionality reduction. Clustering is then performed using the Gaussian Mixture Model (GMM), which is fitted using the Expectation-Maximization (EM) algorithm, and the transformed results from all three dimensionality reduction techniques are compared.

### 2. Dataset

| Sample | Alcohol | MalicAcid | Ash  | Alcalinity | Magnesium | Phenols |
|--------|---------|-----------|------|------------|-----------|---------|
| 1      | 14.23   | 1.71      | 2.43 | 15.6       | 127       | 2.8     |
| 2      | 13.2    | 1.78      | 2.14 | 11.2       | 100       | 2.65    |
| 3      | 13.16   | 2.36      | 2.67 | 18.6       | 101       | 2.8     |
| 4      | 14.37   | 1.95      | 2.5  | 16.8       | 113       | 3.85    |
| 5      | 13.24   | 2.59      | 2.87 | 21         | 118       | 2.8     |
| 6      | 14.2    | 1.76      | 2.45 | 15.2       | 112       | 3.27    |
| 7      | 14.39   | 1.87      | 2.45 | 14.6       | 96        | 2.5     |
| 8      | 14.06   | 2.15      | 2.61 | 17.6       | 121       | 2.6     |
| 9      | 14.83   | 1.64      | 2.17 | 14         | 97        | 2.8     |
| 10     | 13.86   | 1.35      | 2.27 | 16         | 98        | 2.98    |

### 3. Methodology

- Load the wine chemical-measurement dataset into a Pandas DataFrame.
- Standardize all six numerical features using StandardScaler so that features measured on different scales (e.g., Magnesium in mg vs. Phenols in small decimals) are treated equally.
- Apply Principal Component Analysis (PCA) to obtain two orthogonal components that capture maximum variance.
- Apply Factor Analysis (FA) to model the observed variables as linear combinations of latent (hidden) factors plus noise.
- Apply Independent Component Analysis (ICA) to separate the data into statistically independent components.

- Fit a Gaussian Mixture Model (GMM) — trained internally with the Expectation-Maximization (EM) algorithm — on the PCA-reduced data to cluster the wine samples into two groups.
- Compare the two-dimensional embeddings produced by PCA, FA, and ICA side-by-side, coloured by the GMM cluster assignment.

#### 4. Python Program

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA, FactorAnalysis, FastICA
from sklearn.mixture import GaussianMixture

# 1. Dataset
data = {
    "Sample": range(1, 11),
    "Alcohol": [14.23,13.20,13.16,14.37,13.24,14.20,14.39,14.06,14.83,13.86],
    "MalicAcid": [1.71,1.78,2.36,1.95,2.59,1.76,1.87,2.15,1.64,1.35],
    "Ash": [2.43,2.14,2.67,2.50,2.87,2.45,2.45,2.61,2.17,2.27],
    "Alcalinity": [15.6,11.2,18.6,16.8,21.0,15.2,14.6,17.6,14.0,16.0],
    "Magnesium": [127,100,101,113,118,112,96,121,97,98],
    "Phenols": [2.80,2.65,2.80,3.85,2.80,3.27,2.50,2.60,2.80,2.98]
}
df = pd.DataFrame(data)
print("Dataset:\n", df)

features = ["Alcohol","MalicAcid","Ash","Alcalinity","Magnesium","Phenols"]
X = df[features].values

# 2. Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# 3. PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("\nPCA Explained Variance Ratio:", pca.explained_variance_ratio_)
```



```

# 4. Factor Analysis
fa = FactorAnalysis(n_components=2, random_state=42)
X_fa = fa.fit_transform(X_scaled)

# 5. Independent Component Analysis (ICA)
ica = FastICA(n_components=2, random_state=42, max_iter=1000)
X_ica = ica.fit_transform(X_scaled)

# 6. Gaussian Mixture Model clustering (EM Algorithm) on PCA-reduced data
gmm = GaussianMixture(n_components=2, random_state=42)
gmm_labels = gmm.fit_predict(X_pca)
df["GMM_Cluster"] = gmm_labels
print("\nGMM Cluster Assignments:\n", df[["Sample", "GMM_Cluster"]])

# 7. Compare PCA, FA, ICA visually (side-by-side)
fig, axes = plt.subplots(1, 3, figsize=(13, 4.2))
titles = ["PCA", "Factor Analysis", "ICA"]
transforms = [X_pca, X_fa, X_ica]
colors_map = np.array(["black", "gray"])

for ax, title, Xt in zip(axes, titles, transforms):
    ax.scatter(Xt[:, 0], Xt[:, 1], c=colors_map[gmm_labels], s=90,
               edgecolors="black", marker="o")
    for i, txt in enumerate(df["Sample"]):
        ax.annotate(str(txt), (Xt[i, 0], Xt[i, 1]), fontsize=8, xytext=(4,4), textcoords="offset points")
    ax.set_title(title)
    ax.set_xlabel("Component 1")
    ax.set_ylabel("Component 2")
    ax.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.savefig("dimensionality_reduction_comparison.png", dpi=150)
plt.close()

# 8. GMM cluster visualization on PCA components
plt.figure(figsize=(6, 4.5))
colors = ["black", "gray"]
markers = ["o", "s"]
for i in range(2):

```

```

plt.scatter(X_pca[gmm_labels == i, 0], X_pca[gmm_labels == i, 1],
            s=90, c=colors[i], marker=markers[i], label=f"Cluster {i+1}", edgecolors="black")
plt.title("Gaussian Mixture Model Clustering (on PCA components)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.savefig("gmm_clusters.png", dpi=150)
plt.close()

df.to_csv("wine_gmm_output.csv", index=False)
print("\nSaved plots: dimensionality_reduction_comparison.png, gmm_clusters.png")

```

## 5. Program Output

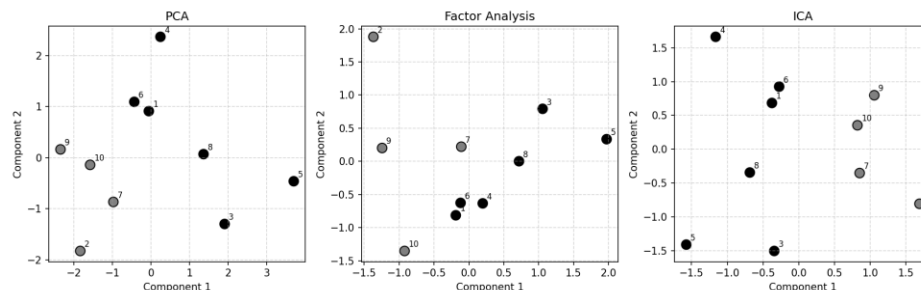
Console Output:

PCA Explained Variance Ratio: [0.5283 0.2274]

GMM Cluster Assignments:

|   | Sample | GMM_Cluster |
|---|--------|-------------|
| 0 | 1      | 0           |
| 1 | 2      | 1           |
| 2 | 3      | 0           |
| 3 | 4      | 0           |
| 4 | 5      | 0           |
| 5 | 6      | 0           |
| 6 | 7      | 1           |
| 7 | 8      | 0           |
| 8 | 9      | 1           |
| 9 | 10     | 1           |

## 6. Screenshots / Visual Output



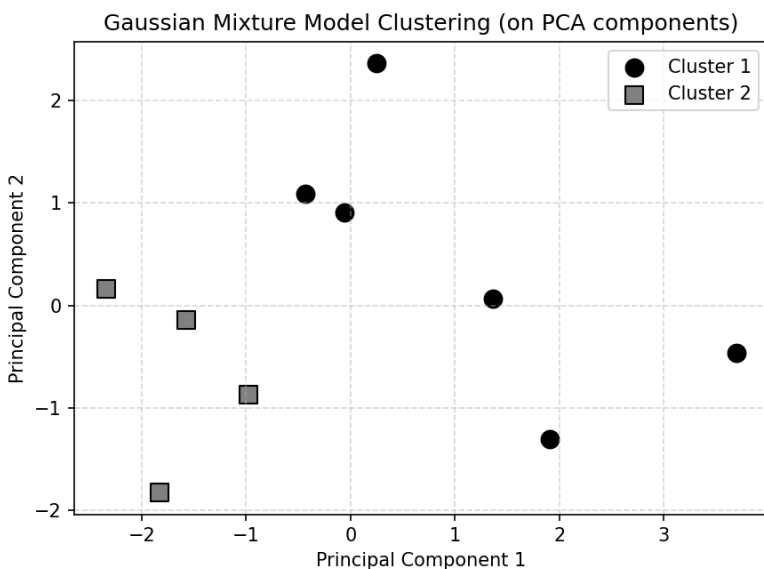


Fig 2.2: Gaussian Mixture Model (EM Algorithm) clustering of wine samples on the PCA components

## 7. Analysis and Insights

PCA captures the directions of maximum variance and shows the first component explaining about 53% of the total variance, with the second component adding about 23%, together giving a reasonable two-dimensional summary of the six chemical attributes. Factor Analysis produces a similar but not identical layout, since it explicitly models measurement noise separately from the shared latent factors, making it more suitable when some variables are noisier than others. ICA produces components that are statistically independent rather than merely uncorrelated, which can reveal hidden source signals in the chemical composition that PCA and FA do not separate as cleanly. The Gaussian Mixture Model, fitted using the EM algorithm on the PCA components, groups the ten wine samples into two probabilistically distinct clusters, reflecting differing chemical profiles (for example, higher Phenols and Alkalinity in one cluster versus more moderate values in the other).

## 8. Conclusion

PCA, Factor Analysis, and ICA each reduce the wine dataset's dimensionality using a different underlying assumption — maximum variance, latent-factor modelling, and statistical independence, respectively — and produce broadly consistent but distinct two-dimensional projections. The Gaussian Mixture Model, using soft probabilistic assignments through the EM algorithm, successfully clusters the wine samples into two chemically distinct groups, demonstrating that dimensionality reduction followed by GMM clustering is an effective pipeline for exploratory analysis of chemical composition data.